FROM SEARCH AND LOGIC TO LEARNING SYSTEMS

Faculty of DS & AI
Autumn semester, 2026

Trong-Nghia Nguyen

Business AI Lab

2026-11

Content

- **What we have learned, and where it stops**
- Machine Learning
- The perceptron
- How a network learns
- Deep Learning
- Putting it together

What we have learned, and where it stops

Seven lectures in one slide

Family	Methods	Example problems
Uninformed search	BFS, DFS	8-puzzle, route finding
Informed search	Greedy, A*, hill climbing	8-puzzle with Manhattan distance
Adversarial search	Minimax, alpha-beta, MCTS	Tic-Tac-Toe, chess, Go
Propositional logic	CNF, resolution (Robinson), chaining	"If it rains, the road is wet"
First-order logic	predicates, quantifiers, unification	"All dogs bark" $\Rightarrow$ "Rex barks"

What all of them have in common

The **rules**, the **heuristic** $h(n)$ and the **evaluation function** $EVAL(s, p)$ were all **written by a human**. The machine only executes them.

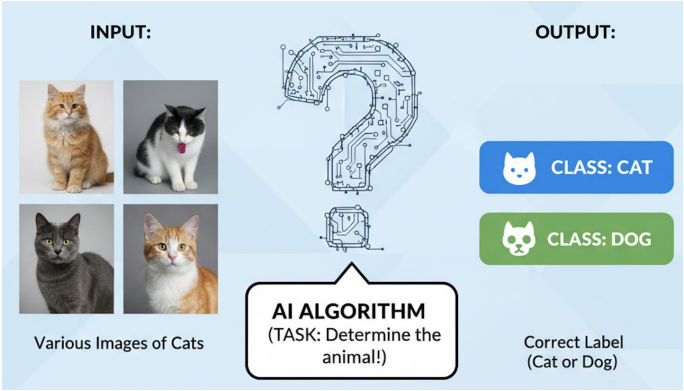
What we have learned, and where it stops

A task we cannot solve yet

“What is in this picture — a cat or a dog?”

- **Task**: decide whether an image shows a cat or a dog
- **Input**: a digital image, e.g. 256×256 pixels, RGB
- **Output**: one label, “cat” or “dog”

A three-year-old does this instantly. Let us try our tools.



What we have learned, and where it stops

Four attempts, four failures

× **Attempt 1 — state space search**

- A state = one complete image
- $256 \times 256 = 65{,}536$ pixels, each with 256^3 colours
- Number of states $= (256^3)^{65{,}536} \approx 10^{473{,}479}$
- 8-puzzle had $9! = 362{,}880$ states

× **Attempt 2 — A* with a heuristic**

- What is $h(n)$ = "distance to cat"?
- Manhattan distance worked because the goal state was *known*. Here it is not

× **Attempt 3 — greedy / evaluation function**

- What makes an image "more cat-like"?
- Nobody can write that formula down

× **Attempt 4 — logical rules**

- $\forall x\,((PointyEars(x) \wedge Whiskers(x)) \Rightarrow Cat(x))$
- An image is a grid of numbers — there is no $PointyEars$ predicate in it
- And every rule has exceptions: folded-ear cats, whiskered dogs

What we have learned, and where it stops

The common cause — and the way out

Why all four attempts failed

Each one needed a **human** to write down, in advance, what a cat *is* — and nobody can, because we do not know the rule we use ourselves.

A different idea

Do not write the rule. **Show the machine thousands of labelled examples** and let it find the rule itself.

- learn patterns from examples, instead of following written rules
- generalise to new, unseen images
- improve when given more data

$\Rightarrow$ **Machine Learning**: systems that learn from data

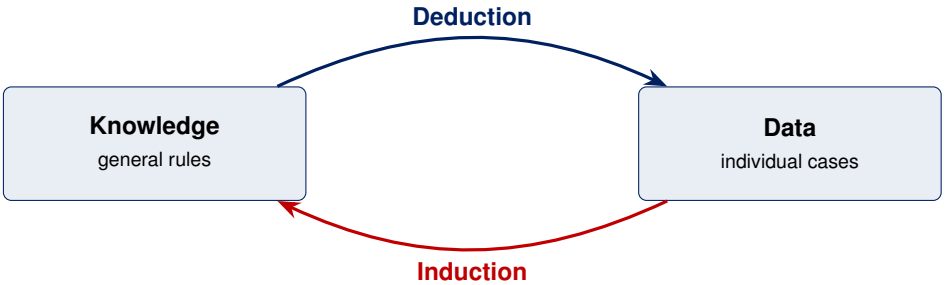
Content

- What we have learned, and where it stops
- **Machine Learning**
- The perceptron
- How a network learns
- Deep Learning
- Putting it together

Machine Learning

Two directions: deduction and induction

$$\forall x\,(Dog(x) \Rightarrow Barks(x)),\ Dog(Rex) \models Barks(Rex)$$



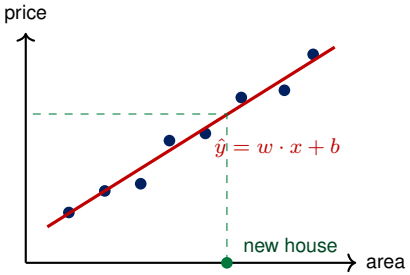
The price of going the other way

Deduction is **guaranteed**; induction is only **probable** — the next dog may be silent.

Machine Learning

What “learning” actually means

A first example: predicting house prices

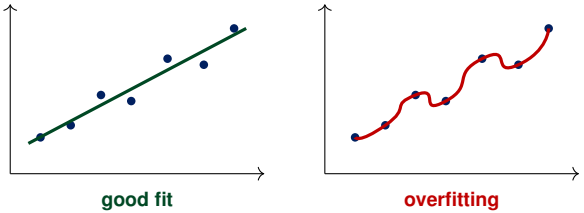


- **Data**: pairs (area, price) of houses already sold
- **Model**: a straight line $\hat{y} = w \cdot x + b$
- **Learning** = choosing the two numbers w and b so the line fits the dots
- **Using it**: for a new house, read the price off the line

The key shift: we did not program the relationship between area and price — we programmed a **procedure for finding** it.

Machine Learning

Generalisation and overfitting



- Split the data: **training set** to fit the model, **test set** to judge it
- **Generalisation**: doing well on data *never seen* during training — the real goal
- **Overfitting**: the model memorises the training points, including their noise, and fails on new data

A student who memorises last year's exam paper, versus one who understands the subject.

Machine Learning

Types of machine learning problems

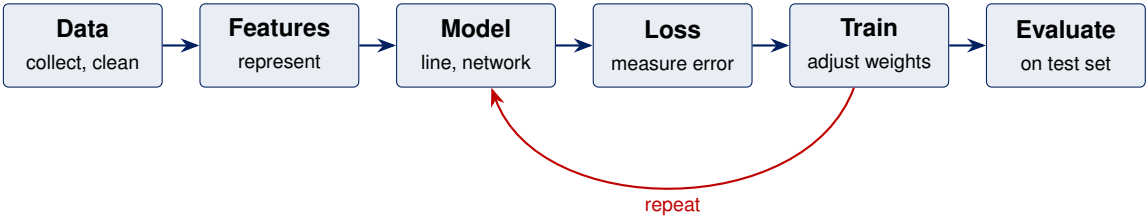
exam

Problem	Data you need	Output	Example
Classification	inputs **with** labels	a discrete label	spam / not spam
Regression	inputs **with** labels	a real number	house price
Clustering	inputs **without** labels	groups	customer segmentation
Dimensionality reduction	inputs **without** labels	fewer features	data visualisation
Reinforcement learning	rewards from an environment	a policy (what to do)	game playing, robots

- **Supervised** = rows 1–2 (labels given); **Unsupervised** = rows 3–4 (no labels)
- Reinforcement learning is neither: no correct answer per example, only a reward — the closest relative of the search problems of Lecture 4

Machine Learning

The shape of every ML project



- The two steps in the middle are where this lecture goes next: **which model?** (perceptron) and **which loss?**
- Deep learning will later remove the **Features** box — the network builds its own

Content

- What we have learned, and where it stops
- Machine Learning
- **The perceptron**
- How a network learns
- Deep Learning
- Putting it together

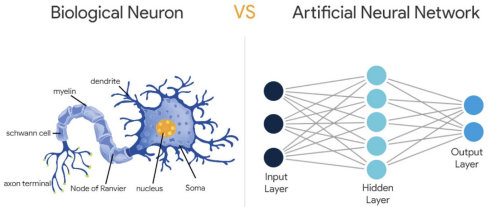
The perceptron

One artificial neuron

A **perceptron** is a **single artificial neuron**: each input is multiplied by a **weight**, a **bias** is added, and it fires if the total reaches 0.

$$y = \begin{cases} 1 & \text{if } w_1 x_1 + \cdots + w_n x_n + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

Biological neuron	Perceptron
strength of a synapse	weight w_i
cell body adds the signals	weighted sum
fires above a threshold	activation



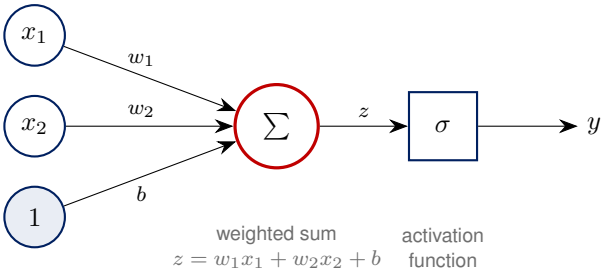
1943 McCulloch & Pitts model one neuron; 1958 Rosenblatt adds a learning rule and names it the *perceptron*.

Inspired by the brain, not a simulation of it

Real neurons fire *spikes over time*, and a brain has ~ 86 billion of them.

The perceptron

Anatomy of a perceptron



- **Weights** w_i: how important each input is — these are the numbers that get **learned**
- **Bias** b: how easy it is to fire
- **Activation** σ: turns the score z into an output

  step function (0 or 1) in 1958; smooth functions such as ReLU today

What it really is

A perceptron draws one **straight line** and asks which side of it the input lies on.

The perceptron

Bridge 1: a perceptron is a logic gate

The weights and the bias are just numbers — so **we may choose them**.

- Choose them well and the perceptron computes exactly the connectives of propositional logic
- This is the original result of McCulloch & Pitts (1943): networks of threshold units compute **logical functions**

Gate	w_1	w_2	b
$x_1 \wedge x_2$	$+1$	$+1$	-2
$x_1 \vee x_2$	$+1$	$+1$	-1
$\neg(x_1 \wedge x_2)$	-1	-1	$+1$
$\neg x_1$	-1	—	0

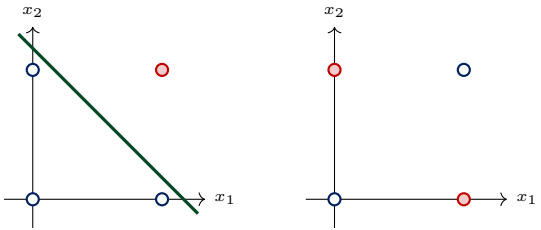
Check $\wedge$ yourself ($w_1 = w_2 = 1, b = -2$)

$$(0,0) \to -2 \to 0 \quad (0,1) \to -1 \to 0 \quad (1,0) \to -1 \to 0 \quad (1,1) \to 0 \to 1 \ \checkmark$$

The perceptron

The problem that stopped AI for 15 years

exam



filled = output 1, empty = output 0

- XOR = “exactly one of the two” — the **“but not both”** pattern you translated in the midterm
- No single straight line can separate the two filled corners from the two empty ones
- A perceptron draws only one line $\Rightarrow$ it **cannot** compute XOR

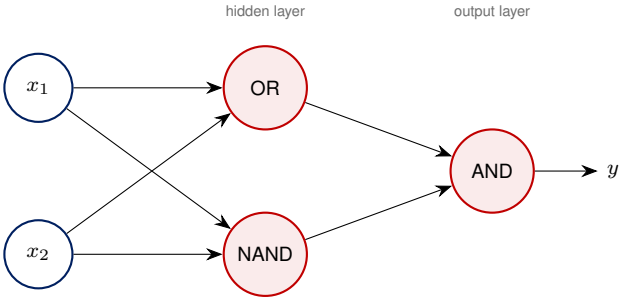
1969 — the first AI winter

Minsky & Papert prove this in *Perceptrons*; funding for neural networks freezes.

The perceptron

One hidden layer solves it — and it is just CNF

$$XOR \equiv (P \vee Q) \wedge \neg(P \wedge Q)$$

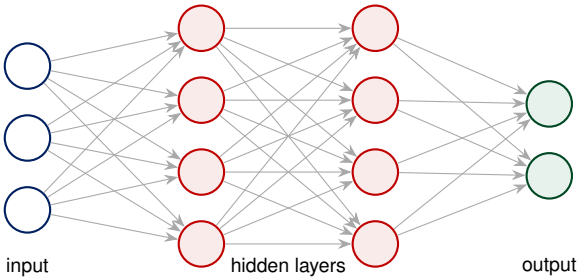


- The **hidden layer** computes the two clauses
- The **output layer** joins them with $\wedge$
- A multi-layer network is a CNF formula whose weights are **learned**

x_1	x_2	OR	NAND	y
0	0	0	1	0
0	1	1	1	1
1	0	1	1	1
1	1	1	0	0

The perceptron

The multi-layer perceptron (MLP)



- Many perceptrons side by side form a **layer**; layers stacked form a **network**
- Each layer bends the decision boundary a little more — straight lines become curves
- Enough hidden units can approximate **any** continuous function

But one question remains

A network of this size has thousands of weights. **How do we find values for all of them?**

Content

- What we have learned, and where it stops
- Machine Learning
- The perceptron
- **How a network learns**
- Deep Learning
- Putting it together

How a network learns

Bridge 2: from heuristic to loss function

exam

	Search and logic (weeks 1–7)	Machine learning
The scoring function	$h(n)$, $f(n) = g(n) + h(n)$, $EVAL(s, p)$	the **loss** $L(w)$
It scores	how good a **state** is	how wrong a set of **weights** is
Where it comes from	**a human writes it** (Manhattan distance, material count)	**computed from data**: compare prediction with the correct label
Goal	reach a state with $h = 0$	find weights with L as small as possible
What we search over	states of the problem	numbers $w_1, w_2, \ldots$

The whole idea in one sentence

We could not write a heuristic for “is this a cat”, but we *can* write a formula for **how wrong an answer is** — and then let the machine make that number small.

How a network learns

Two loss functions you will meet

Regression — Mean Squared Error

$$L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- y_i the true price, $\hat{y}_i$ the predicted one
- squaring: errors in both directions count, and large errors hurt much more

Classification — Cross-Entropy

$$L = -\frac{1}{n}\sum_{i=1}^{n}\log \hat{p}(\text{correct class})$$

- the model outputs a *probability* for each class
- confident and right $\rightarrow$ loss near 0; confident and wrong $\rightarrow$ very large loss

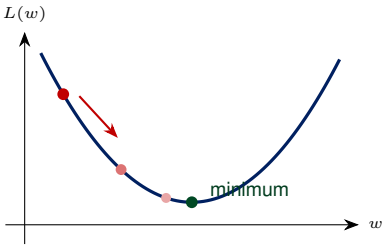
Why not simply count the mistakes?

Accuracy moves in jumps, so it cannot say *which direction* improves the weights.

How a network learns

Bridge 3: gradient descent is hill climbing

exam



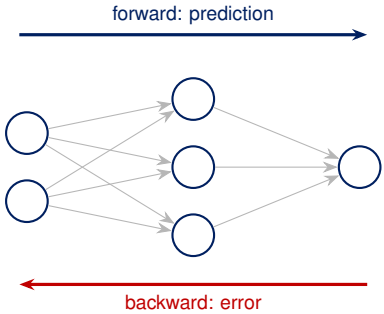
	Hill climbing	Gradient descent
Search space	states	weights w
One move	go to a neighbour	$w \leftarrow w - \eta \nabla L$
Direction	best neighbour	steepest downhill
Step size	one move	learning rate η
Stops at	local optimum	local optimum

- ∇L (the *gradient*) just says: which way is uphill. We step the other way
- η too large: we jump over the valley. Too small: training takes forever

Same algorithm as Lecture 2 — including the same weakness: it can get stuck in a local optimum.

How a network learns

Back-propagation: how the error reaches every weight



- **Forward**: push the input through the layers, get a prediction, compute the loss
- **Backward**: send the error back layer by layer, so each weight learns how much *it* contributed
- Then every weight takes one gradient-descent step

Why it mattered (Lecture 1, Phase 6)

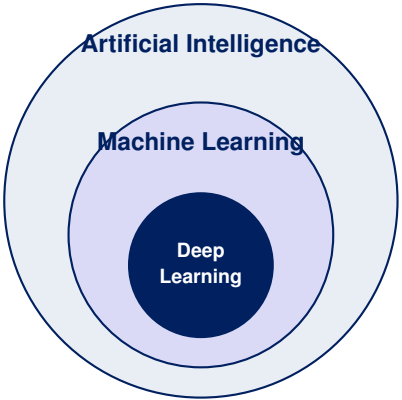
1986 — the PDP group (Rumelhart, Hinton, McClelland) popularises **back-propagation**: multi-layer networks become trainable, and the first AI winter ends.

Content

- What we have learned, and where it stops
- Machine Learning
- The perceptron
- How a network learns
- **Deep Learning**
- Putting it together

Deep Learning

Deep learning in one picture



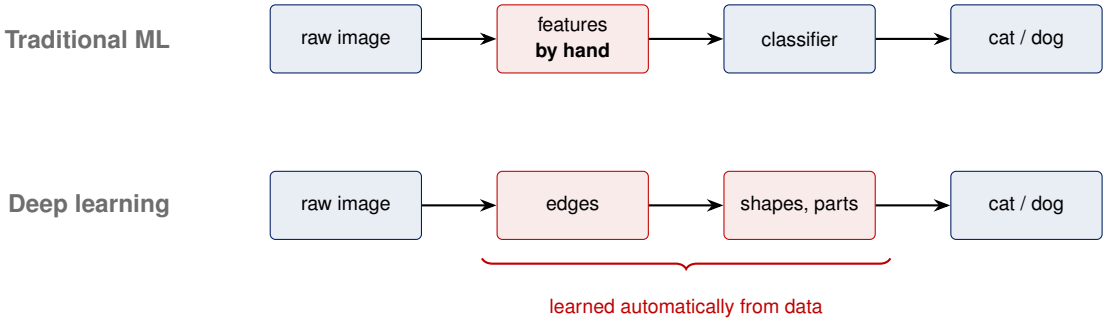
- **AI**: any technique that makes machines behave intelligently — *includes search and logic*
- **ML**: learns the rules from data instead of being given them
- **DL**: machine learning carried out by neural networks with **many** hidden layers

Position of this course

Weeks 1–7 were the outer ring. Today we opened the two inner ones.

Deep Learning

What the extra layers buy us



- Traditional ML needed an expert to define “edges”, “corners”, “textures” by hand
- A deep network discovers that hierarchy itself: pixels $\rightarrow$ edges $\rightarrow$ shapes $\rightarrow$ objects

Deep Learning

Architectures and the problems they fit

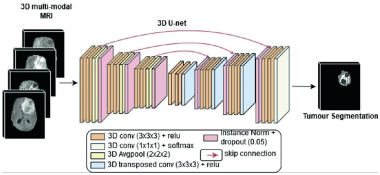
exam

Architecture	Built for	Key idea	Typical task
MLP	tables of numbers	fully connected layers	credit scoring
CNN	images	small filters slide over the image	image classification
RNN / **LSTM**	sequences	a memory of what came before	time series, speech, EEG
Transformer	text	attention: every token looks at every other	translation, chatbots
GAN / **diffusion**	images, audio	two models compete, or noise is removed step by step	image generation

- The **shape of the data** chooses the architecture: a grid, a sequence or a plain table
- All of them use the same machinery: weights, a loss, gradient descent, back-propagation

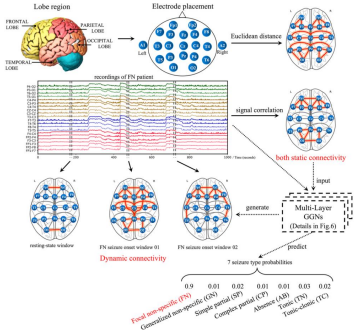
Deep Learning

Deep learning at work



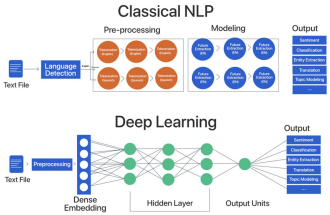
Computer vision

tumour segmentation in MRI



Natural language

translation, chatbots



Signal processing

EEG and biomedical signals

Deep Learning

What deep learning costs

Limitations

- **Data hungry**: often needs thousands to millions of labelled examples
- **Expensive**: large models need GPUs, time and energy
- **Hard to explain**: millions of weights, no readable reason for a decision
- **No guarantees**: it is induction — a confident answer can still be wrong

Compare with what we learned first

A resolution proof can be **printed and checked line by line**. It tells you *why* the conclusion holds.

- In medicine, law or credit scoring, that explanation may matter more than a few points of accuracy
- Explainable AI is an active research area, trying to give networks that property back

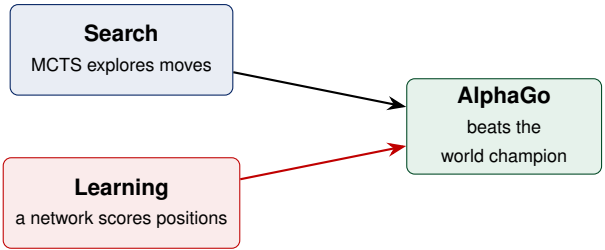
Content

- What we have learned, and where it stops
- Machine Learning
- The perceptron
- How a network learns
- Deep Learning
- **Putting it together**

Putting it together

Symbolic and learning AI are not rivals

AlphaGo (2016) — both halves of this course in one system



- In Lecture 4 the function $EVAL(s, p)$ had to be **written by hand** — impossible for Go
- AlphaGo keeps the search, but **learns** $EVAL$ from millions of games
- Search supplies look-ahead; learning supplies the intuition

The same pattern appears when an LLM is allowed to call a solver or a planner.

Putting it together

The whole course on one page

	Symbolic AI (weeks 1–7)	Learning-based AI (today)
Knowledge from	a human writing rules	data
Reasoning	deduction: rules $\rightarrow$ conclusion	induction: examples $\rightarrow$ rule
Scoring function	$h(n)$, $EVAL(s, p)$ — hand-written	loss $L(w)$ — computed from data
Search	over states (A*, minimax, hill climbing)	over weights (gradient descent)
Strength	exact, explainable, needs no data	handles images, speech, text
Weakness	needs rules someone can state	needs data; hard to explain

Neither one is "the" AI. Strong systems use both.

Putting it together

Key takeaways

- Search and logic stop where **nobody can write the rule** — that is where learning starts
- A **perceptron** is a weighted sum plus a threshold; with the right weights it is a logic gate
- A single perceptron draws **one line**, so it cannot do XOR; a **hidden layer** can — that is just CNF with learned weights
- A **loss function** plays the role of the heuristic, but is computed from data
- **Gradient descent** is hill climbing in weight space; **back-propagation** is what makes it possible for many layers
- **Deep learning** = many layers, so the features are learned rather than hand-crafted

Next courses: Machine Learning, Deep Learning — where each box of today's pipeline becomes a full chapter.

Thank you!

You’re now ready to explore the exciting world of AI!